

ScolarApp

Application Mobile de Gestion Scolaire

Rapport Technique et Fonctionnel

Auteur	ALIBO CORENTIN
--------	----------------

Formation	L3 Informatique
-----------	-----------------

Annee	2026
-------	------

Technologies	Flutter / Laravel / MySQL
--------------	---------------------------

Depot GitHub	roor-killla/mobileapp-2026 (branche alibo)
--------------	--

1. Presentation du projet

ScolarApp est une application mobile de gestion scolaire developpee dans le cadre du cours de L3 Informatique. Elle s'inspire du fonctionnement de Pronote et propose trois espaces distincts : un espace professeur, un espace administrateur et un espace etudiant. L'application repose sur une architecture client-serveur avec un frontend Flutter et un backend Laravel connecte a une base de donnees MySQL.

1.1 Architecture technique

Composant	Technologie	Role
Frontend	Flutter (Dart)	Interface mobile Android
Backend	Laravel 11 (PHP 8.4)	API REST - Architecture MVC
Base de donnees	MySQL 8.4	Stockage des donnees
Serveur local	Laragon (Apache)	Hebergement du backend
Notifications	flutter_local_notifications	Alertes nouveaux devoirs
Chatbot	API Anthropic (Claude)	Assistant IA pour etudiants
Securite	Bcrypt (Laravel Hash)	Hashage des mots de passe

L'emulateur Android communique avec le backend via l'adresse 10.0.2.2 qui correspond au localhost de la machine hote. Laragon sert le backend depuis C:\laragon\www\backend-api.

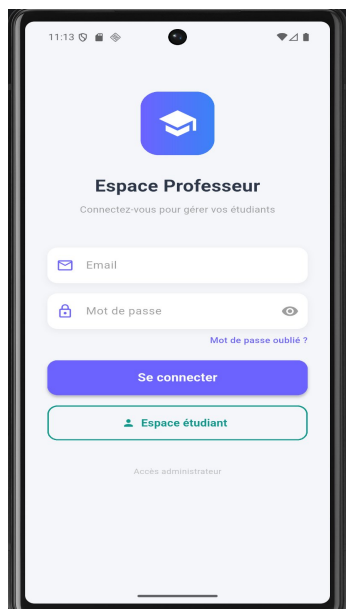
1.2 Structure du projet

Dossier	Contenu
mobile/lib/screens/	Tous les ecrans Flutter (login, dashboard, notes, devoirs...)
mobile/lib/models/	Modeles de donnees (Etudiant, Note, Professeur...)
mobile/lib/services/	Services API, Session, Notifications
backend-api/app/Models/	Modeles Laravel (Eloquent ORM)
backend-api/app/Http/Controllers/	Controllers API REST
backend-api/database/migrations/	Migrations de la base de donnees
backend-api/routes/api.php	Definition de toutes les routes API

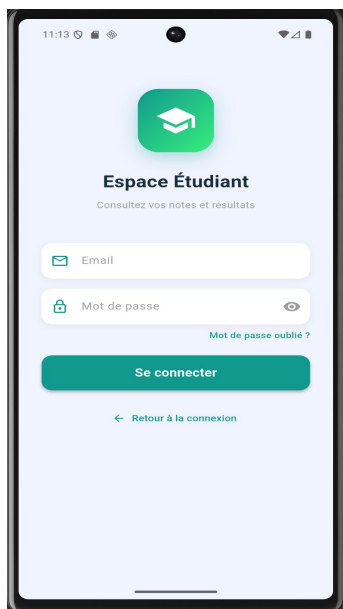
2. Fonctionnalités de l'application

2.1 Ecrans de connexion

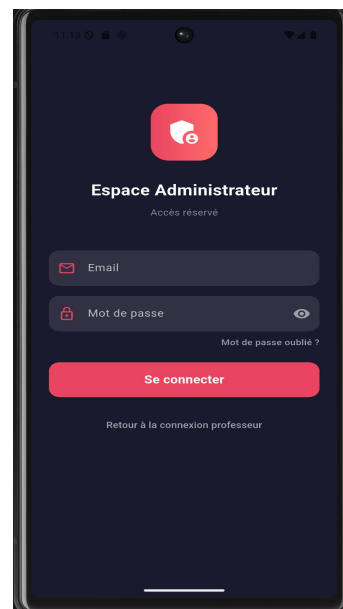
L'application propose trois écrans de connexion distincts, chacun avec une identité visuelle propre. Chaque espace inclut une fonctionnalité de réinitialisation de mot de passe par génération d'un code temporaire.



Espace Professeur (violet)



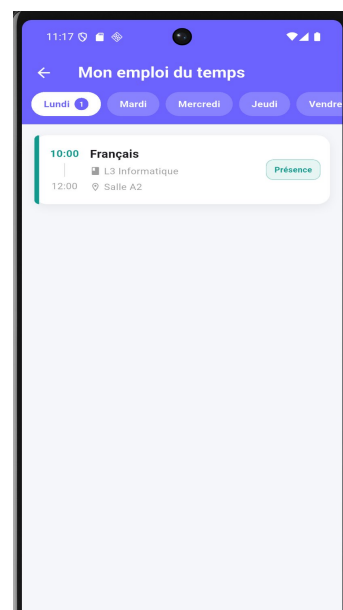
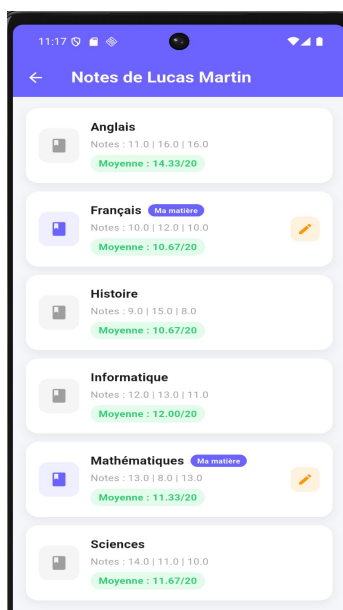
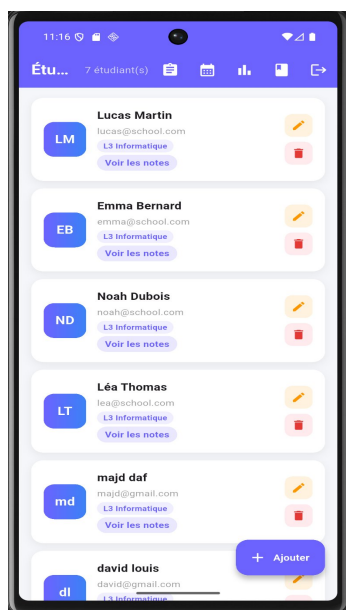
Espace Etudiant (vert)

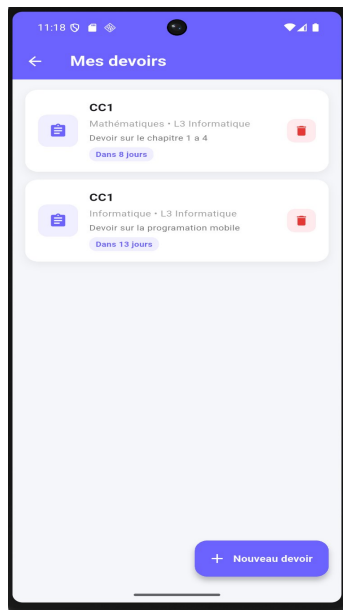


Espace Admin (sombre)

2.2 Espace Professeur

L'espace professeur (couleur violette) permet de gérer les étudiants, les notes, l'emploi du temps et les devoirs. Le professeur voit la liste complète des étudiants avec leur classe, peut consulter et modifier les notes de ses matières (max 2 matières, max 3 notes par matière), déclarer sa présence ou absence pour chaque cours de son emploi du temps, et créer des devoirs avec date limite pour ses classes.





L'écran des devoirs permet au professeur de créer des travaux à rendre en spécifiant la matière, la classe concernée, une description détaillée et la date limite. Un système de code couleur indique l'urgence : rouge pour moins de 2 jours, orange pour moins de 5 jours, violet pour les échéances plus lointaines. Les étudiants reçoivent une notification locale automatique lors de l'ajout d'un nouveau devoir par un professeur.

2.3 Espace Administrateur

L'espace administrateur (theme sombre rouge) centralise la gestion des professeurs et des emplois du temps. L'admin peut créer et supprimer des professeurs, leur assigner des matières (max 2 par professeur), et gérer les créneaux de cours pour chaque classe en spécifiant le jour, les horaires, la salle et le professeur.

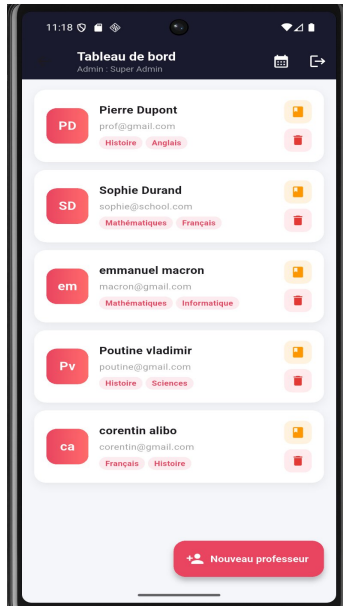
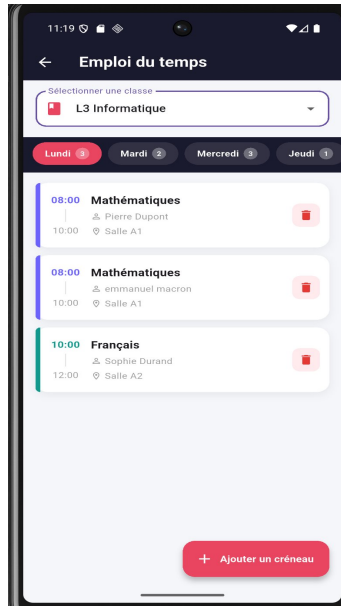


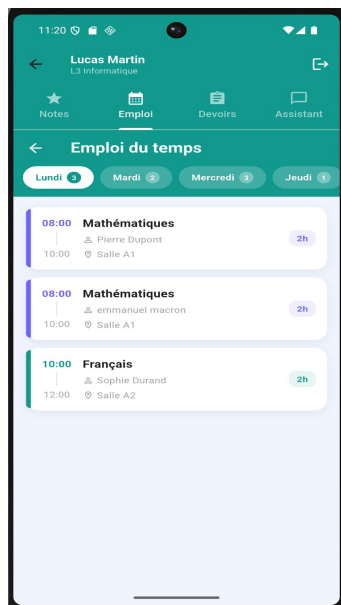
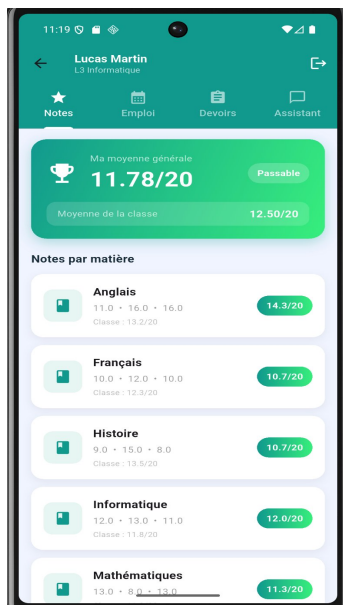
Tableau de bord Admin



Gestion emploi du temps

2.4 Espace Etudiant

L'espace étudiant (couleur verte) offre quatre onglets : Notes, Emploi du temps, Devoirs et Assistant IA. Le tableau de bord affiche la moyenne générale avec mention et la compare à la moyenne de la classe. Chaque matière affiche les trois notes, la moyenne individuelle et la moyenne de la classe. L'emploi du temps est consultable par jour avec les horaires, la salle et le nom du professeur.



2.5 Systeme de classes

L'application gere trois classes : L1 Informatique, L2 Informatique et L3 Informatique. Chaque etudiant est assigne a une classe. Les moyennes sont calculees par classe et par matiere, ce qui permet a l'etudiant de se situer par rapport a l'ensemble de sa promotion.

3. Securite

3.1 Hashage des mots de passe

Les mots de passe ne sont jamais stockes en clair dans la base de donnees. La fonction Hash::make() de Laravel utilise l'algorithme bcrypt pour transformer un mot de passe en une chaine illisible et irreversible. Lors de la connexion, Hash::check() compare le mot de passe saisi avec le hash stocke sans jamais dechiffrer ce dernier. Le champ password est declare dans le tableau \$hidden de chaque modele Laravel, ce qui garantit qu'il ne sera jamais retourne dans les reponses JSON de l'API.

3.2 Separation des roles

Chaque role (admin, professeur, etudiant) possede son propre ecran de connexion et ses propres routes API. Un etudiant ne peut pas acceder aux routes admin ou professeur. Cette separation est implementee directement au niveau du routage Laravel.

3.3 Reinitialisation de mot de passe

En cas d'oubli de mot de passe, l'utilisateur peut demander une reinitialisation en saisissant son email. Le systeme genere un mot de passe temporaire aleatoire (3 lettres + 4 chiffres) et le hache avant de le stocker en base. Le mot de passe temporaire est affiche une seule fois a l'ecran.

4. Difficulte rencontree

4.1 Conflit de dossiers Laragon

La principale difficulte rencontree lors du developpement concernait la configuration de l'environnement de developpement. Laragon possede son propre dossier de travail situe dans C:\laragon\www\, tandis que le projet etait edite dans un dossier sur le bureau (OneDrive). Les modifications apportees aux fichiers du backend ne prenaient pas effet car Laragon lisait les fichiers depuis son propre dossier, pas depuis celui qui etait modifie. Cela generait des erreurs difficiles a diagnostiquer car aucun message clair n'indiquait la source du probleme. La solution a ete de toujours travailler directement dans C:\laragon\www\backend-api et d'y copier les fichiers modifies.

4.2 Colonne obsolete dans la base de donnees

La table etudiants contenait initialement une colonne 'note' (valeur globale NOT NULL) qui est devenue inutile apres la mise en place d'un systeme de notes par matiere. Cette colonne bloquait silencieusement toutes les insertions de nouveaux etudiants avec une erreur SQL. La solution a ete de creer une migration Laravel pour supprimer proprement cette colonne obsolete.

4.3 Accent dans le parsing JSON Flutter

Le modele Professeur.dart utilisait json['matieres'] avec accent, alors que l'API Laravel retournait le champ sans accent. Ce probleme silencieux provoquait un affichage vide des matieres dans l'espace administrateur. La correction a consiste a aligner le nom du champ JSON avec ce que retourne reellement l'API.

5. Comptes de test

Professeurs

Nom	Email	Mot de passe	Matieres
Pierre Dupont	prof@gmail.com	password	Histoire, Anglais
Sophie Durand	sophie@school.com	sophie123	Mathematiques, Francais
Emmanuel Macron	macron@gmail.com	test123	Mathematiques, Informatique
Poutine Vladimir	poutine@gmail.com	(reinit.)	Histoire, Sciences
Corentin Alibo	corentin@gmail.com	(reinit.)	Francais, Histoire

Administrateur

Email	Mot de passe
admin@gmail.com	admin123

Etudiants (mot de passe = prenom minuscule + 123)

Nom	Email	Mot de passe	Classe
Lucas Martin	lucas@school.com	lucas123	L3 Informatique
Emma Bernard	emma@school.com	emma123	L3 Informatique
Noah Dubois	noah@school.com	noah123	L3 Informatique
Lea Thomas	lea@school.com	lea123	L3 Informatique
Hugo Petit	hugo@school.com	hugo123	L3 Informatique
Chloe Robert	chloe@school.com	chloe123	L3 Informatique
Antoine Richard	antoine@school.com	antoine123	L3 Informatique

6. Instructions de lancement

6.1 Prerequis

Outil	Usage
Laragon	Serveur Apache + MySQL local
Composer	Gestionnaire de dependances PHP
Flutter SDK	Developpement mobile
Android Studio	Emulateur Android

6.2 Lancement du backend

1. Demarrer Laragon (Start All)
2. Creer la base de donnees backend_api dans HeidiSQL
3. Copier les fichiers backend dans C:\laragon\www\backend-api
4. Ouvrir un terminal dans C:\laragon\www\backend-api et executer :

```
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate
php artisan db:seed
```

6.3 Lancement du frontend Flutter

```
cd mobile
flutter pub get
flutter run
```

Note importante : Le backend doit impérativement etre place dans C:\laragon\www\backend-api et non dans le dossier du projet sur le bureau. Laragon sert uniquement les fichiers depuis son propre dossier www.

7. Bilan et perspectives

7.1 Fonctionnalités réalisées

Fonctionnalité	Espace	Statut
Connexion avec reinitialisation de mot de passe	Tous	Realise
Gestion des etudiants (CRUD)	Professeur	Realise
Gestion des notes par matiere	Professeur	Realise
Emploi du temps avec presence/absence	Professeur	Realise
Systeme de devoirs avec notifications	Professeur / Etudiant	Realise
Gestion des professeurs et matieres	Admin	Realise
Gestion de l'emploi du temps par classe	Admin	Realise
Dashboard notes avec moyenne de classe	Etudiant	Realise
Emploi du temps et devoirs	Etudiant	Realise
Chatbot IA (Assistant)	Etudiant	Realise (sans cle API)
Systeme de classes (L1, L2, L3)	Tous	Realise
Gestion des absences	Etudiant	A venir

7.2 Perspectives d'amélioration

Plusieurs améliorations sont envisageables pour faire évoluer l'application. L'ajout d'une authentification par token JWT ou Laravel Sanctum renforcerait la sécurité de l'API. La mise en place de notifications push Firebase permettrait d'alerter les étudiants même quand l'application est fermée. L'intégration d'un vrai chatbot avec une clé API Anthropic ou Mistral AI rendrait l'assistant pleinement fonctionnel. Enfin, le module de gestion des absences étudiants et la possibilité de modifier la classe d'un étudiant existant sont des fonctionnalités planifiées.